

PRESS RECORD

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

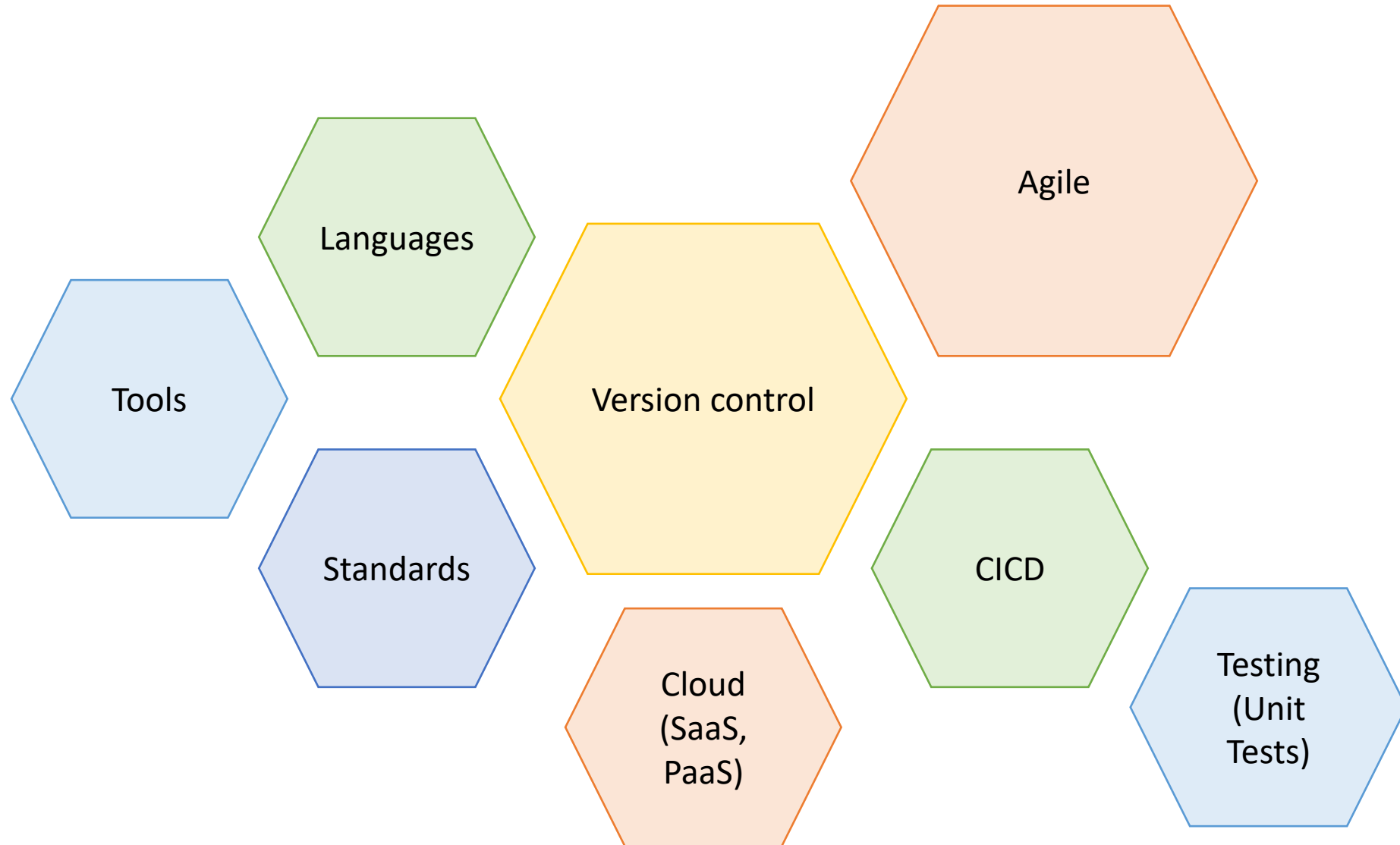
Git and GitLab

COMPX₃₄₁

Before we start ...

- FYI – Industry Input
- FYI – Guest Speakers
- FYI – Soft Schedule

FYI – Industry Input



FYI – Guest Speakers



Sarah Mann
Junior Software developer
NetValue Ltd



Liam Clow
DevOps Engineer
Hamilton City Council

FYI – Soft Schedule

	Lecture 1	Lecture 2	Lecture 3	Assessments
Week 7	Testing w Steve	ANZAC Day	Construction Git and GitLab	
Week 8	Construction Life Cycle Models	Construction Software Quality	Guest Speaker Sarah Mann	CW2 Due CW3 Released
Week 9	Construction Code Quality	Construction Tools and Example	Maintenance An Introduction	
Week 10	Maintenance Analysis and Testing	Maintenance Tools and Example	DevOps An Introduction	CW3 Due CW4 Released
Week 11	DevOps Measuring and Embracing Risk	DevOps CICD Example	Guest Speaker Liam Clow	
Week 12	TBD	TBD	TEST 2	CW4 Due

Git and GitLab

COMPX₃₄₁

Overview

- Version Control
- Git
- GitLab
- UoW GitLab
- Example

Version Control

Version Control

What is Version Control?

- “Version control, also known as source control, is the practice of tracking and managing changes to software code.” ([Bitbucket](#))
- “Version control software keeps track of every modification to the code in a special kind of database. If a mistake is made, developers can turn back the clock and compare earlier versions of the code to help fix the mistake while minimizing disruption to all team members.” ([Bitbucket](#))
- “Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.” ([Git](#))

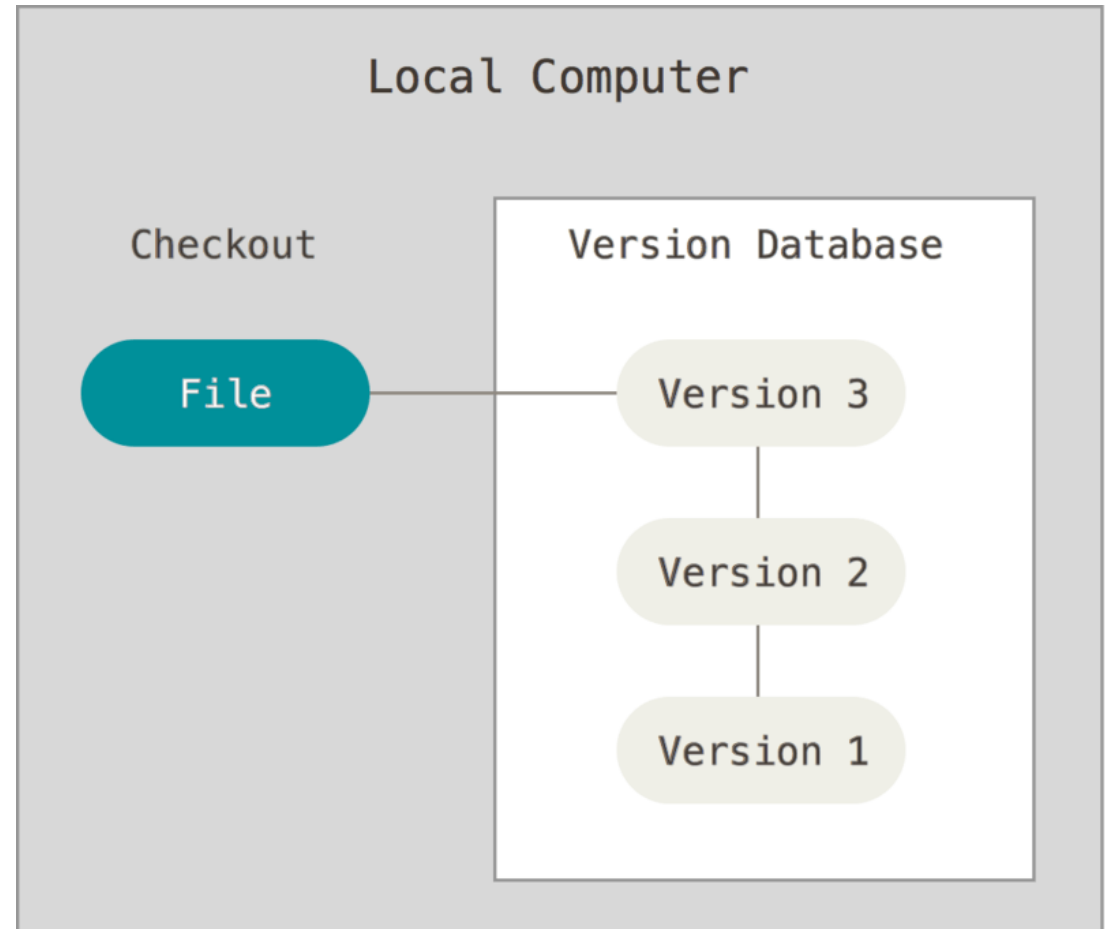
Version Control

Types of Version Control Systems

- Local version control
- Centralized version control
- Distributed version control

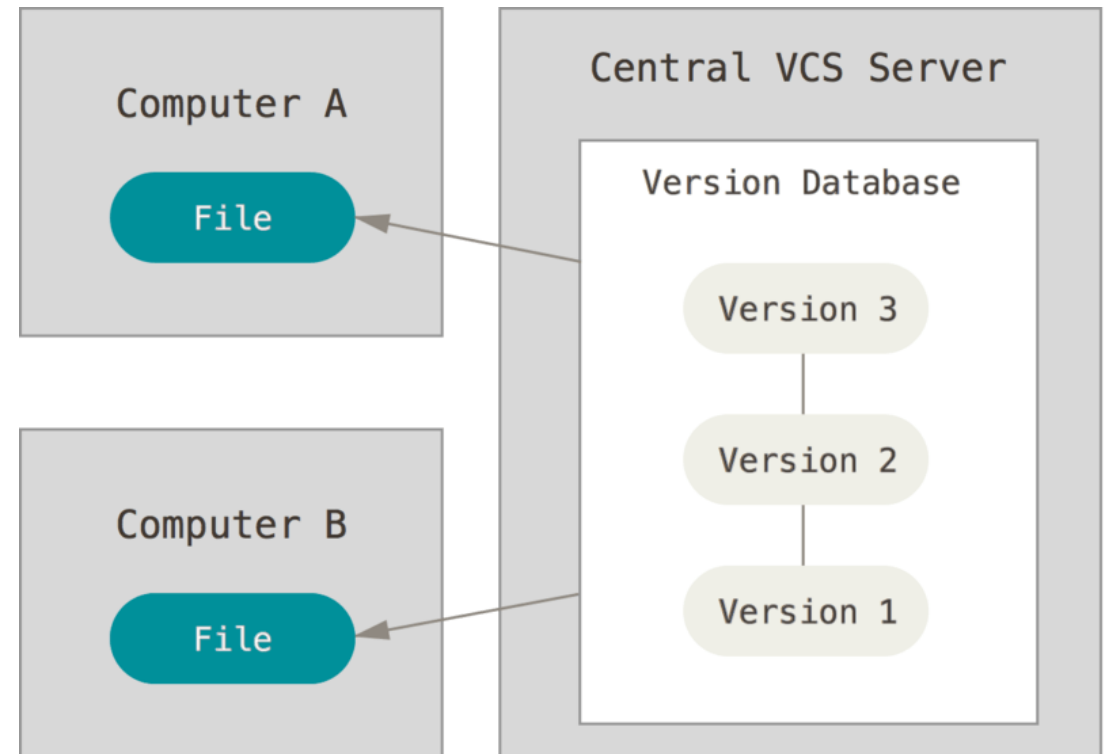
Local Version Control Systems

- Uses a (simple) local database
- Database keeps all file changes
- Called revision control
- Examples: RCS
- ([Git](#))



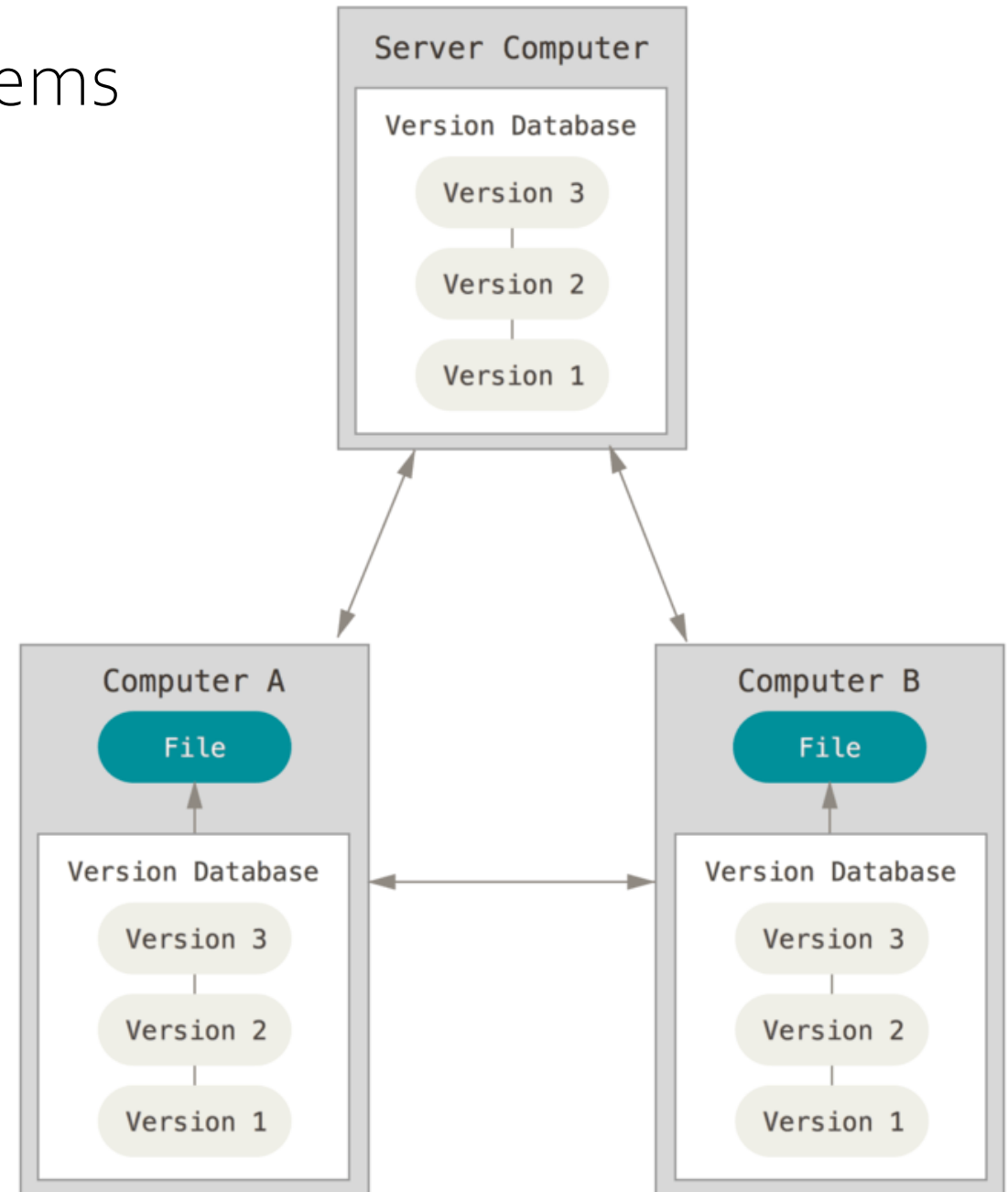
Centralized Version Control Systems

- Enables collaboration
- Single server that holds all versioned files
- Clients “check out” files from the server
- Examples: CVS, Subversion (SVN), and Perforce
- ([Git](#))



Distributed Version Control Systems

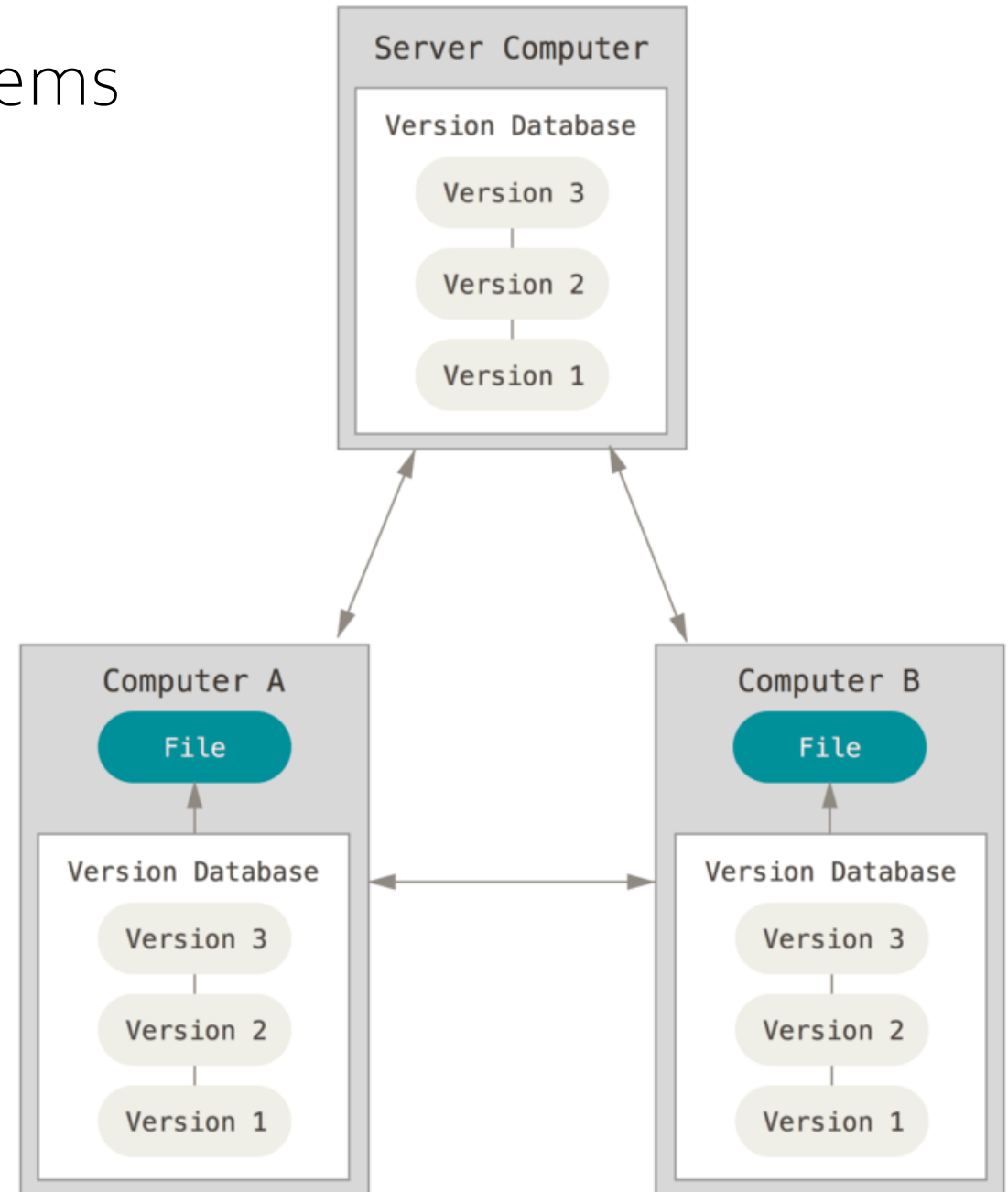
- Client's "clone" the whole repository, including its history.
- Every clone is a full backup of all the data.
- Examples: Git, Mercurial, Bazaar or Darcs
- ([Git](#))



Git

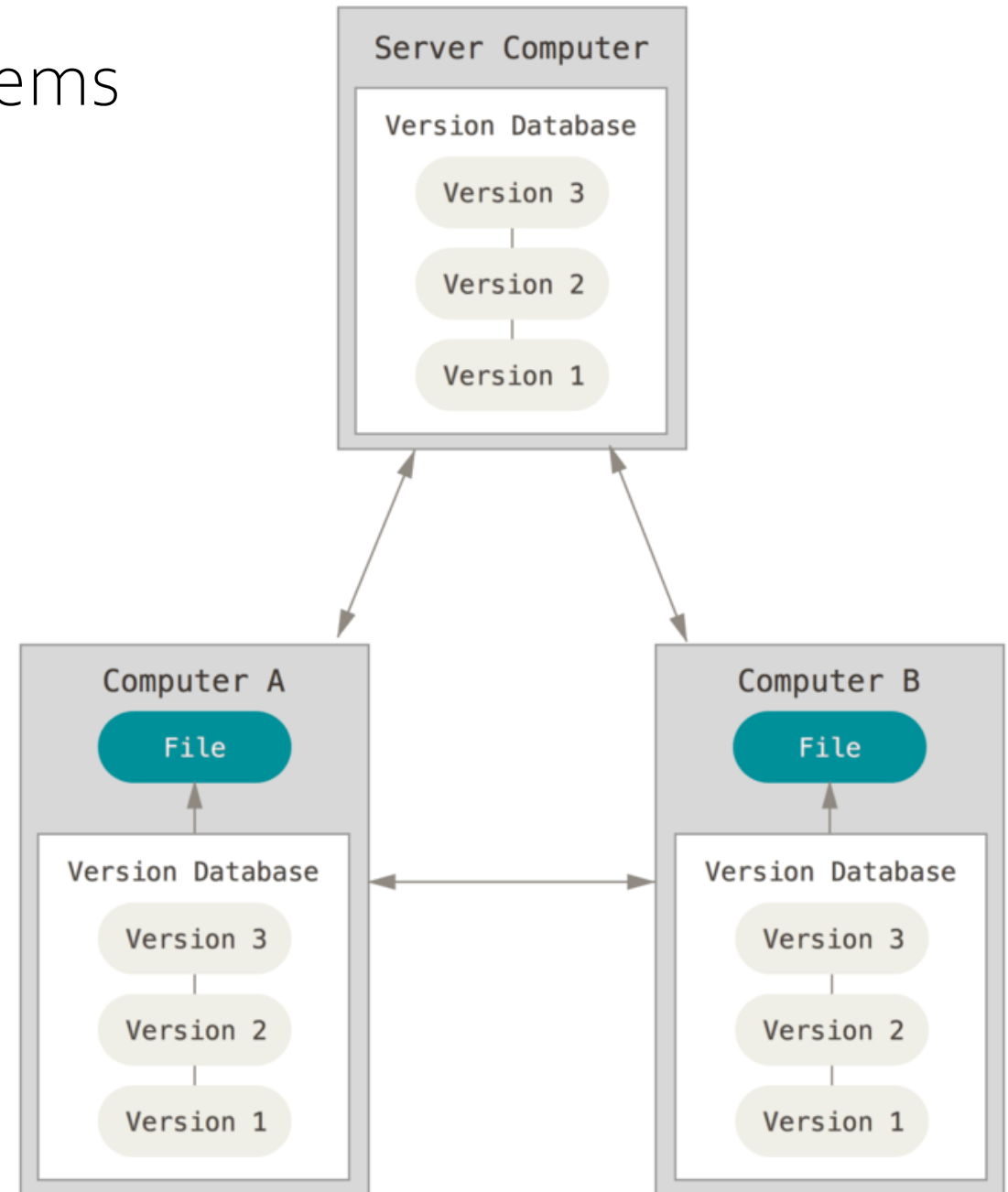
Distributed Version Control Systems

- Client's "clone" the whole repository, including its history.
- Every clone is a full backup of all the data.
- Examples: Git, Mercurial, Bazaar or Darcs
- ([Git](#))



Distributed Version Control Systems

- Client's “clone” the whole repository, including its history.
- Every clone is a full backup of all the data.
- Examples: **Git**, Mercurial, Bazaar or Darcs
- ([Git](#))



Git Stages

Git has three main stages: modified, staged, and committed

Modified:

- You have changed the file but have not staged or commit it yet

Staged:

- You have marked a modified file to go into your next commit

Committed:

- The data has been safely stored in your local database

Working tree, staging area, and Git directory

Working tree:

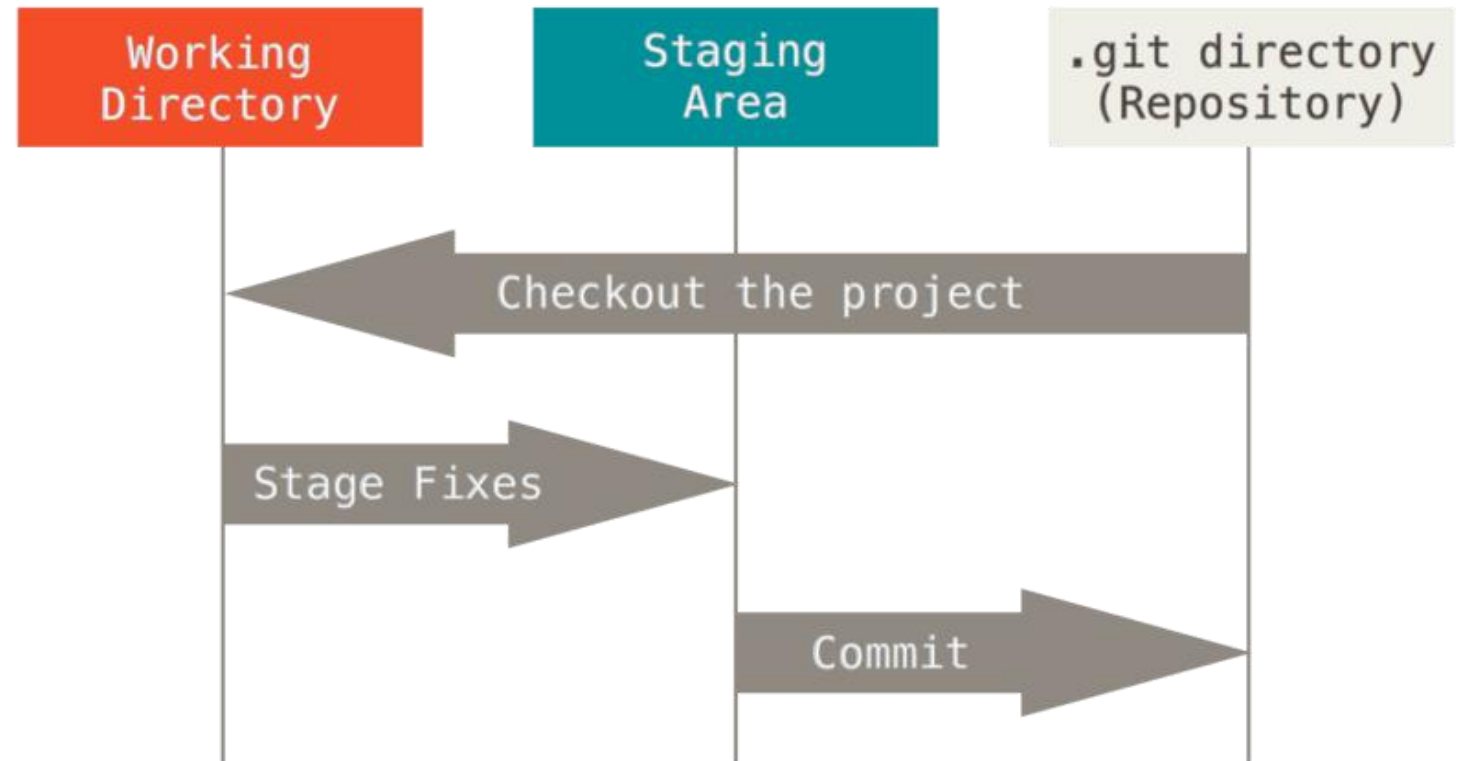
- Single checkout of one version of the project.
- Stored locally on your computer

Staging Area

- A file stored in your Git directory
- Stores info about what goes in your next commit

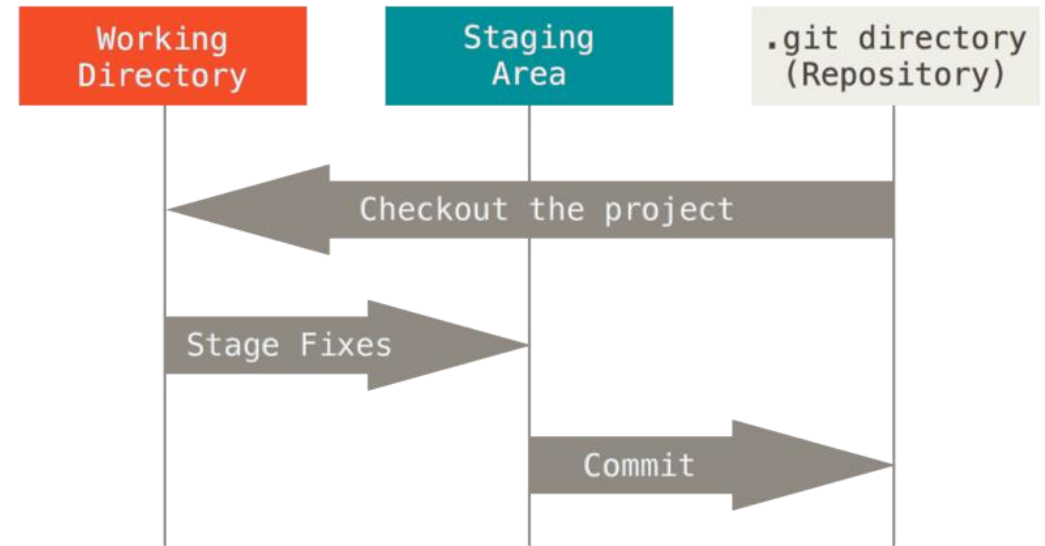
Git Directory

- Where Git stores the metadata and Object database
- Most important part of Git
- This is what is copied when you "clone"



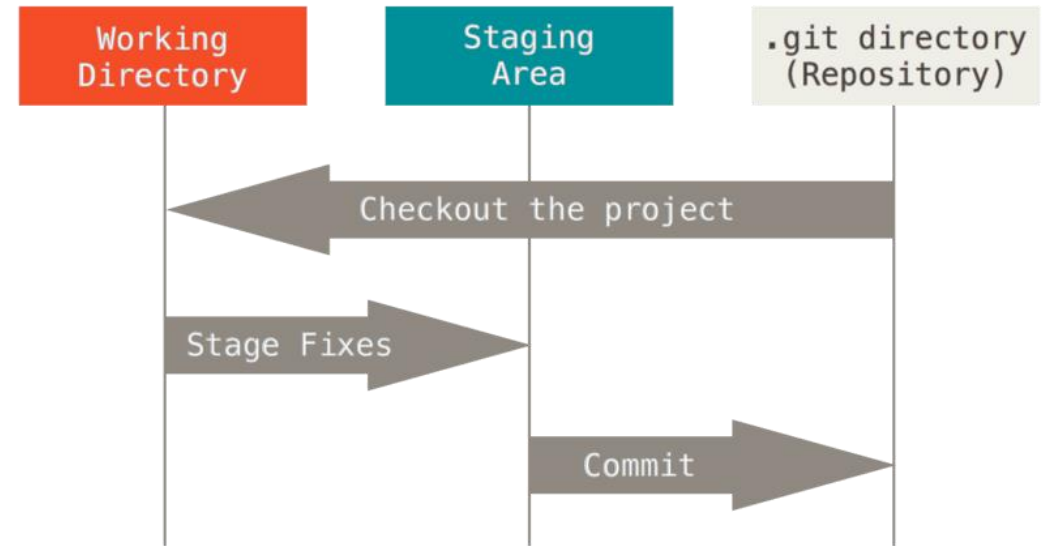
Git Workflow

1. You modify files in your working tree
2. You stage the changes you want to be included in your next commit
3. You do a commit. This takes the files as they are in the staging area and adds them to your .git directory



Git Workflow

“If a particular version of a file is in the Git directory, it’s considered **committed**. If it has been modified and was added to the staging area, it is **staged**. And if it was changed since it was checked out but has not been staged, it is **modified**.”



Git Command Line

- Okay, but how do we actually do this?
- You can use Git from the Command Line
 - But how do we do that?

Git Command Line

- [Installing Git](#)

Git Command Line

Useful Git commands

- `git clone <url>`
- `git checkout <branch>`
- `git pull`
- `git add <files>`
- `git commit -m <"commit message">`
- `git push <branch>`

Git Clone

Useful Git commands

- `git clone <url>`
- `Git clone https://gitlab.com/a4220/mytestrepo.git`

Git Clone

Useful Git commands

- `git clone <url>`
- `Git clone https://gitlab.com/a4220/mytestrepo.git`

GitLab

What is GitLab?

- **Git** is a Distributed Version Control System
- **GitLab** is a Git client / Git management software
- Web-based Git management software that provides free public and private repositories

Version Control
Types

Version Control
Systems

Version Control
Clients

Version Control

Local Version
Control

Centralized Version
Control

Distributed Version
Control

...

...

...

Mercurial



Git



...

Source
tree



GitHub



GitLab



Bitbucket



GitLab

- Using GitLab we can:
 - Create public and/or private repositories
 - Track issues
 - Handle merge requests
 - And more
- Using Git we can:
 - Clone our GitLab repositories
 - Add, commit, push, pull, branch, merge, and so on.

UoW GitLab

What is UoW GitLab?

- The University has a teaching instance of GitLab (actually a few)
- The SCMS instance is found at courses-git.cms.waikato.ac.nz
- You will need to select Waikato AD and sign in with your CMS (Linux) username and password
- Let's take a look ...

Git and GitLab Example

Git and GitLab Example

Example One: Starting out

1. Create a repository (GitLab)
2. Clone the repository (Git clone)
3. Create a Java file and write some code
4. Add changes to the staging area (Git add)
5. Commit changes to the .git directory (Git commit)
6. Update the online GitLab repository (Git push)

Git and GitLab Example

Example Two: Coming back

- | | |
|---|--|
| 1. Create a repository | (GitLab) |
| 2. Clone the repository | (Git clone) |
| 3. Make sure you have the latest version | (Git fetch && git pull) |
| 4. Create a Java file and write some code | |
| 5. Add changes to the staging area | (Git add) |
| 6. Commit changes to the .git directory | (Git commit) |
| 7. Update the online GitLab repository | (Git push) |

Git and GitLab Example

Example Three: Starting out w Branching

1. Create a repository (GitLab)
2. Create a branch (GitLab)
3. Clone the repository (Git clone)
4. Checkout your branch (Git checkout <branch>)
5. Create a Java file and write some code
6. Add changes to the staging area (Git add)
7. Commit changes to the .git directory (Git commit)
8. Update the online GitLab repository (Git push)

Git and GitLab Example

Example Four: Coming back w Branching

- | | |
|---|--|
| 1. Create a repository | (GitLab) |
| 2. Create a branch | (GitLab) |
| 3. Clone the repository | (Git clone) |
| 4. Checkout your branch | (Git checkout <branch>) |
| 5. Make sure you have the latest version | (Git fetch && git pull) |
| 6. Create a Java file and write some code | |
| 7. Add changes to the staging area | (Git add) |
| 8. Commit changes to the .git directory | (Git commit) |
| 9. Update the online GitLab repository | (Git push) |

Git and GitLab Example

Example Five: Merge your branch with Main

1. Checkout your branch (Git checkout <branch>)
2. Make sure you have the latest version (Git fetch && git pull)
3. Make sure the latest Main is on your branch (Git rebase main)
4. Switch to Main (Git checkout main)
5. Merge your branch with Main (Git merge <branch>)
6. Update the online GitLab repository (Git push)

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato